

Software Development Fundamentals (SDF) – I

ODD 2023



Pointers

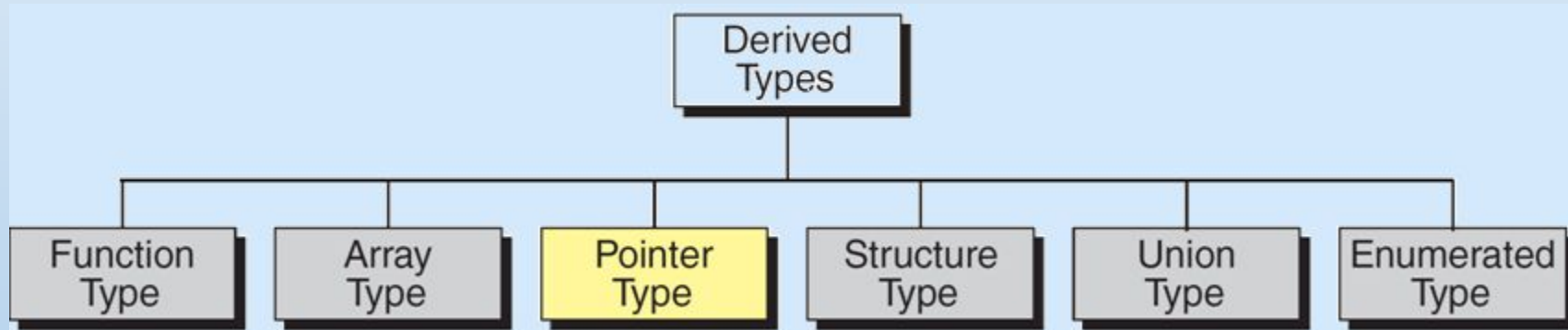
Jaypee Institute of Information Technology (JIIT), Noida

Updated by Amitesh, Dr. Ashish Mishra on 14-09-2023. If any errors or correction is required – kindly report it on Amitesh@mail.jiit.ac.in; Ashish.Mishra@mail.jiit.ac.in

Outline

- Derived Types
- Introduction to Pointers
- Pointer Declaration and Initialization
- Pointer Types
- Pointer pointing to one or more variables
- Pointer Usage and Arithmetic
- Pointers in 1D Array

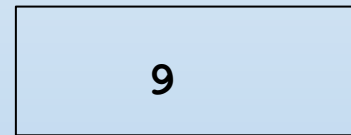
Derived Types



Introduction to Pointers

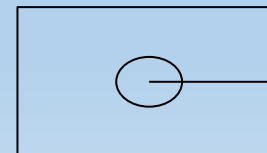
- Contain **memory addresses as their values**
- Normal variables contain a specific value (direct reference)

Normal_variable

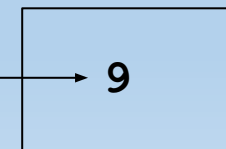


- Pointers contain address of a variable that has a specific value (**indirect reference**)
- Indirection – referencing a pointer value

Pointer_variable



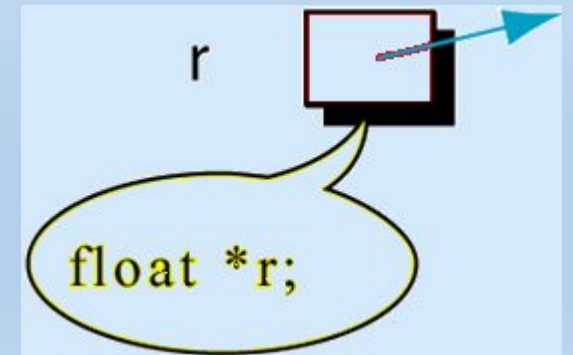
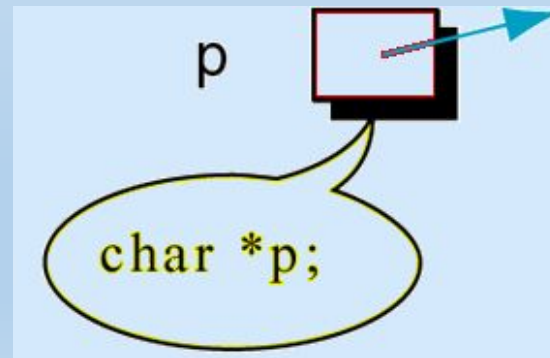
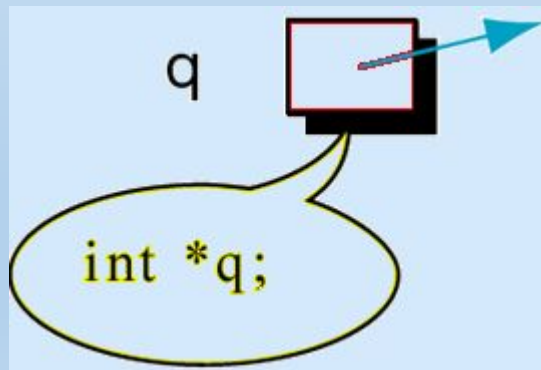
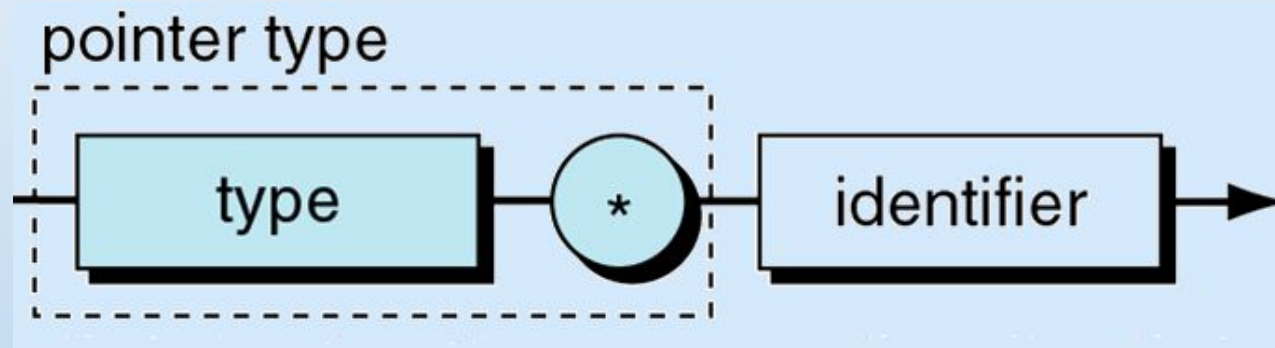
Normal_variable

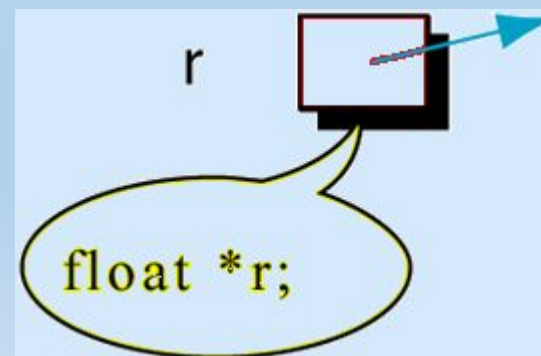
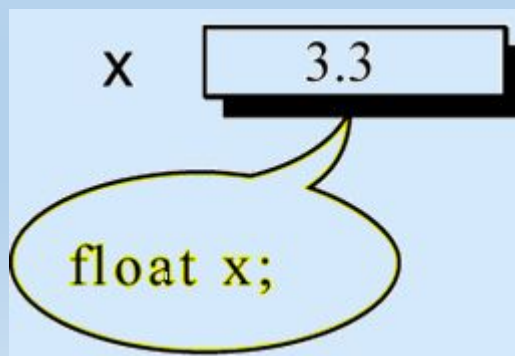
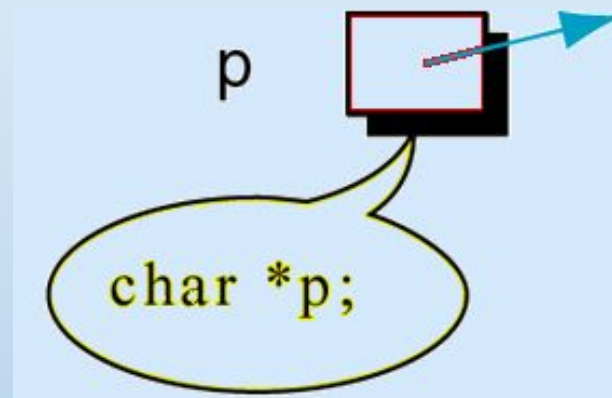
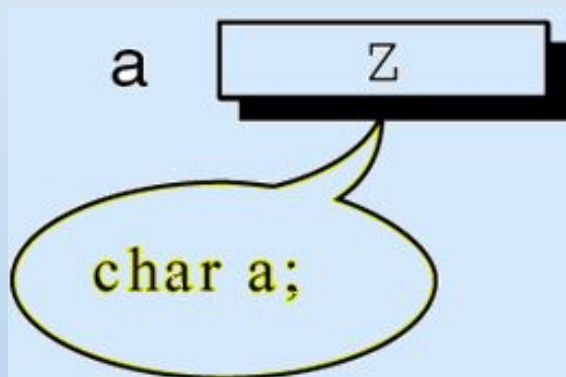
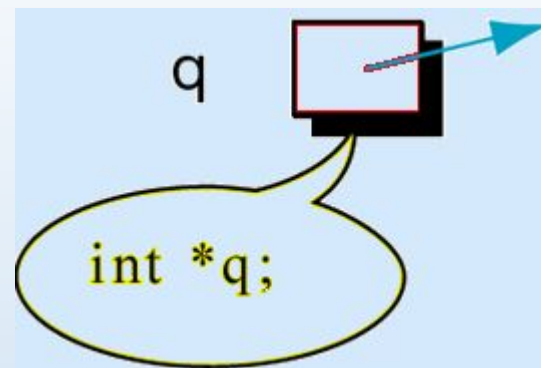
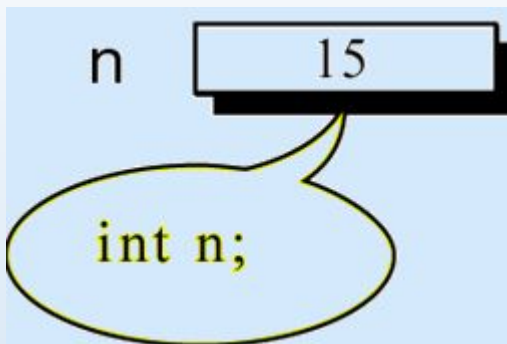


Pointer Declaration and Initialization

- ***** used with **pointer variables**
int *myPtr;
- **Declares a pointer to an int** (pointer of type **int ***)
- Multiple pointers require using a ***** before each variable declaration
int *myPtr1, *myPtr2;
- Can **declare pointers to any data type**
- Initialize **pointers to 0, NULL, or an address**
 - **0** or **NULL** – **points to nothing** (**NULL** preferred)

Pointer Type





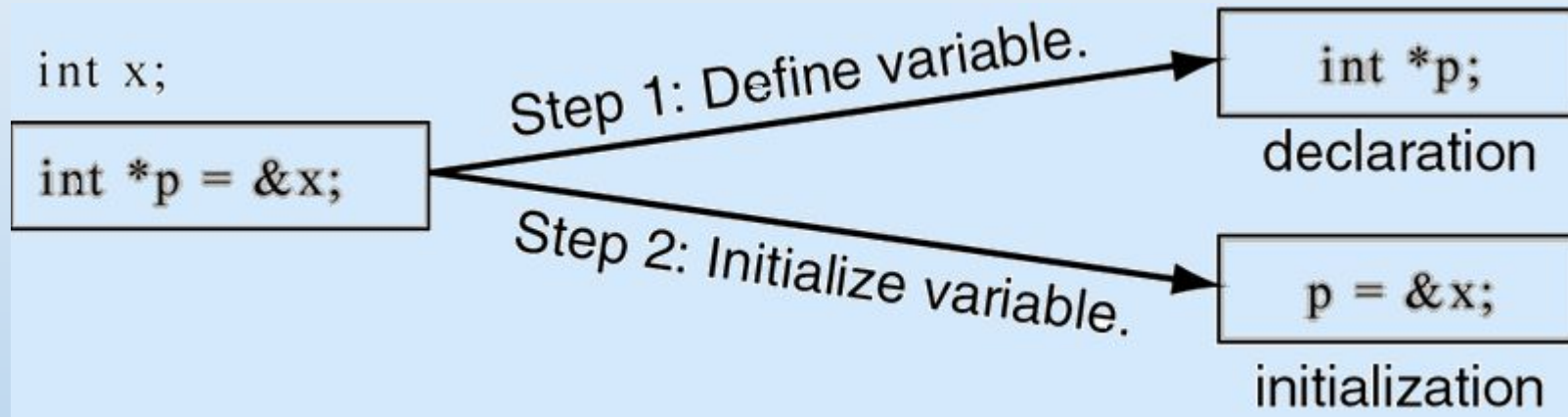
Store address of variable to another variable

```
int n;  
int *q;    // Declares pointer to int  
q = &n;    // address of n is stored in q
```

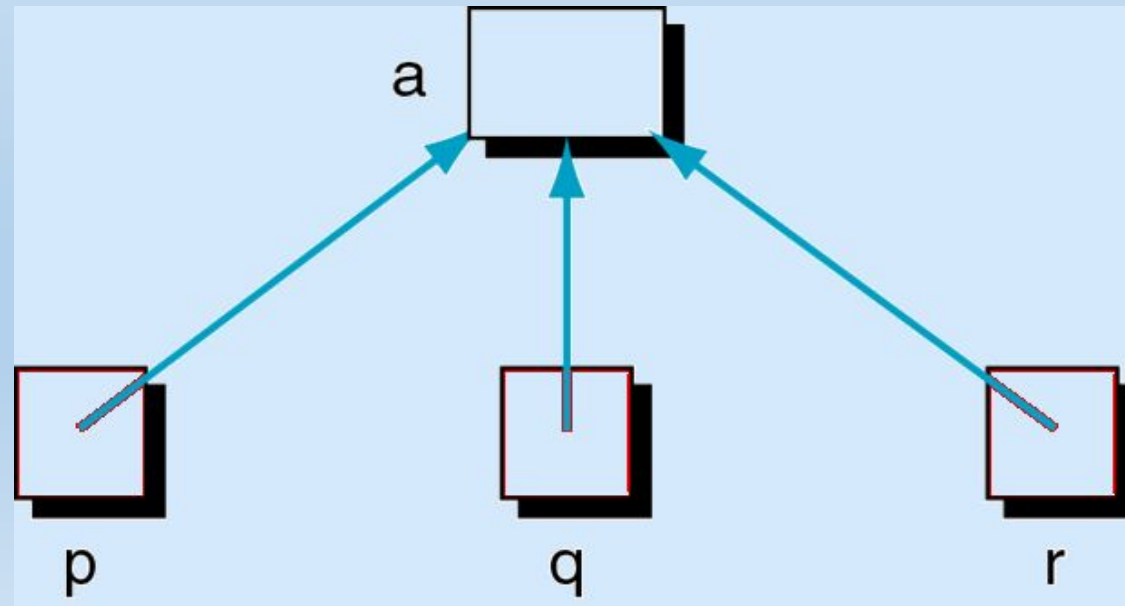
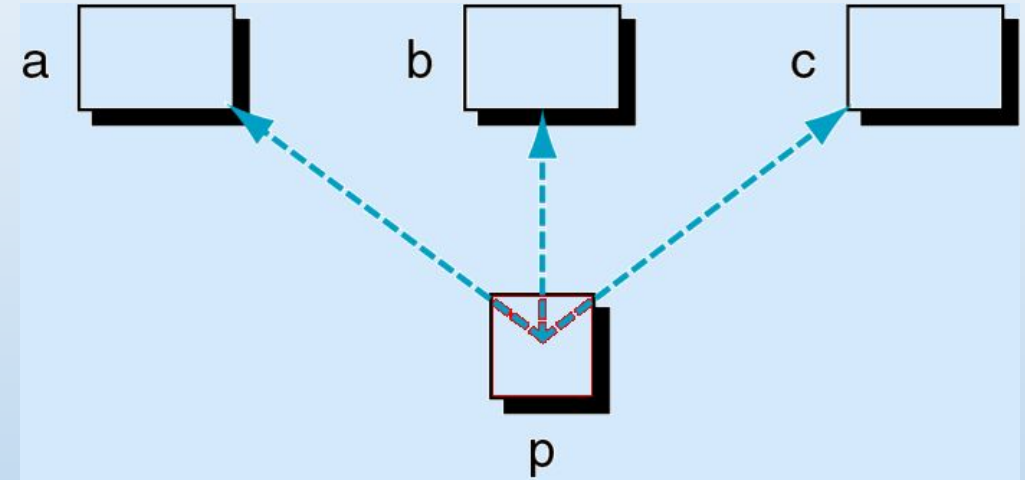
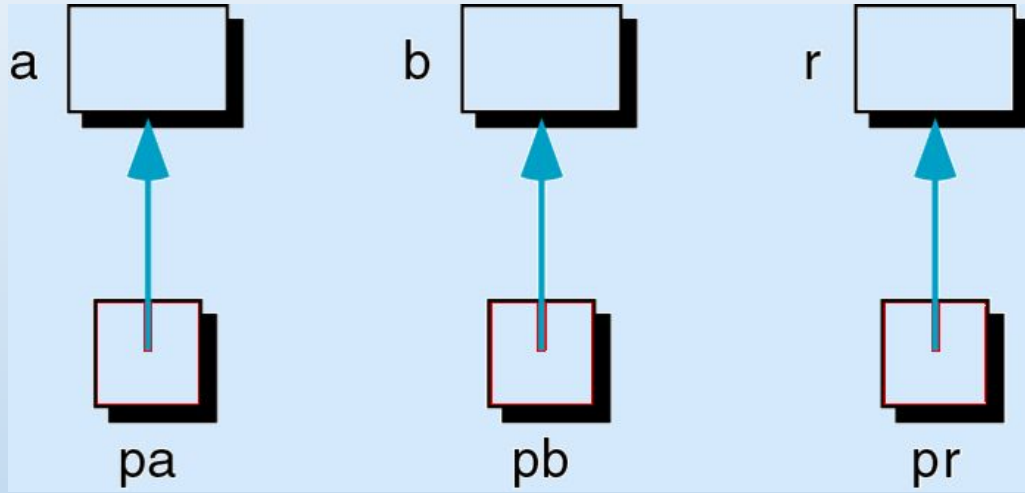
```
char a;  
char *p;   // Declares pointer to char  
p = &a;    // address of a is stored in p
```

```
float x;  
float *r;  // Declares pointer to float  
r = &x;    // address of x is stored in r
```

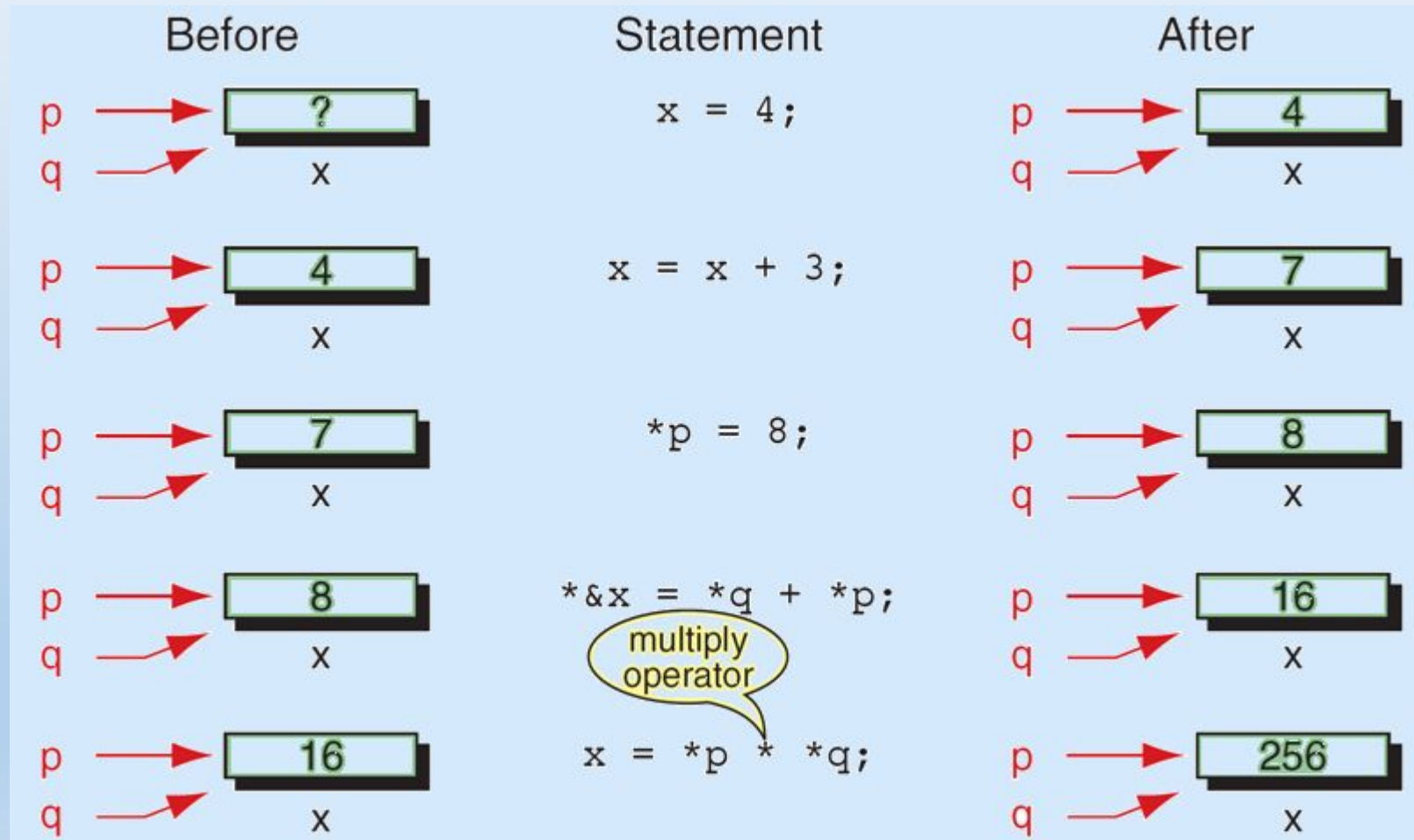
Define and Initialize Pointer variable



Pointer pointing to variable(s)



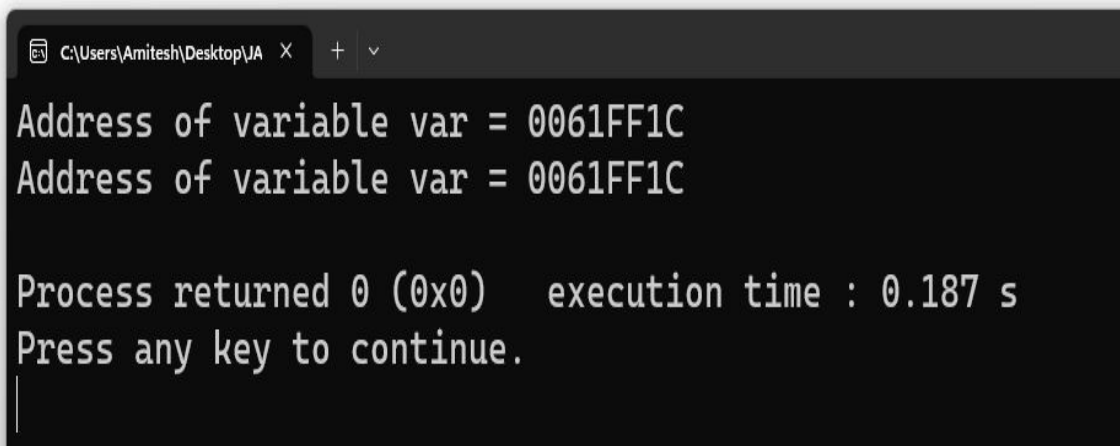
Accessing Variables Through Pointers



Format specifier %p

- This format specifier is used to find the memory location or address at which the value is stored by a variable. It allows to display the address in hexadecimal form.

```
#include<stdio.h>
int main()
{
    int var;
    printf("Address of variable var = %p\n", &var);
    var = 9;
    printf("Address of variable var = %p\n", &var);
    return 0;
}
```



```
C:\Users\Amitesh\Desktop\JA X + v
Address of variable var = 0061FF1C
Address of variable var = 0061FF1C

Process returned 0 (0x0) execution time : 0.187 s
Press any key to continue.
|
```

Demonstration of Pointer

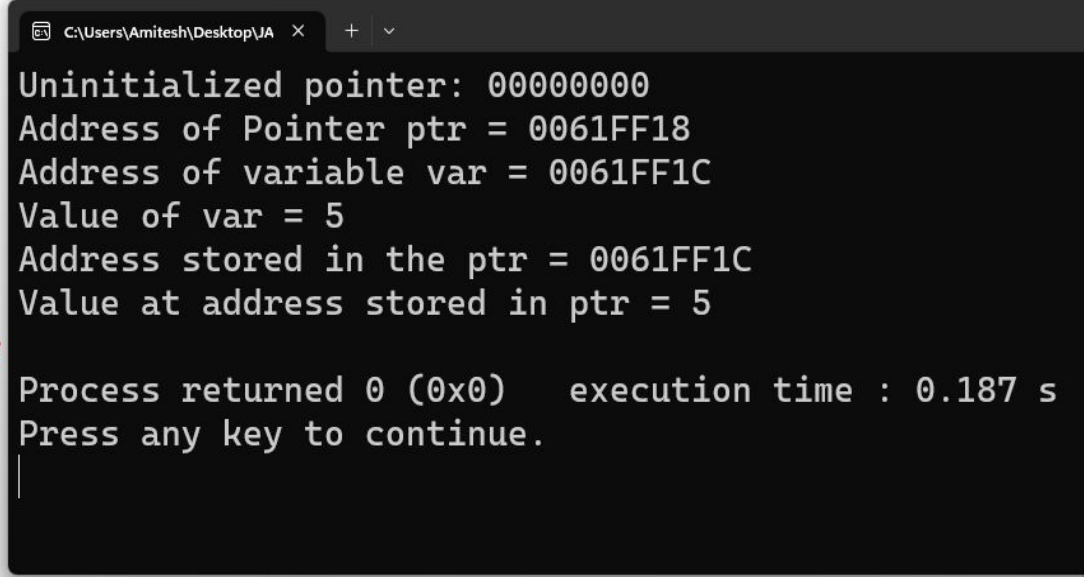
```
#include<stdio.h>
int main()
{
    int var = 0;
    int *ptr;
    printf("Uninitialized pointer: %p\n", ptr);
    printf("Address of Pointer ptr = %p\n", &ptr);

    printf("Address of variable var = %p\n", &var);

    ptr = &var;
    *ptr = 5;

    printf("Value of var = %d\n", var);
    printf("Address stored in the ptr = %p\n", ptr);
    printf("Value at address stored in ptr = %d\n", *ptr);

    return 0;
}
```



```
C:\Users\Amitesh\Desktop\JA
Uninitialized pointer: 00000000
Address of Pointer ptr = 0061FF18
Address of variable var = 0061FF1C
Value of var = 5
Address stored in the ptr = 0061FF1C
Value at address stored in ptr = 5
Process returned 0 (0x0) execution time : 0.187 s
Press any key to continue.
```

Program

```
#include <stdio.h>
int main (void)
{
    int a, b, c;
    int *p, *q, *r;

    a = 6;
    b = 2;

    p = &b;
    q = p;
    r = &c;
```

```
    p = &a;
    *q = 8;

    *r = *p;
    *r = a + *q + *&c;

    printf("%d %d %d\n", a, b, c);
    printf("%d %d %d", *p, *q, *r);
    return 0;
```

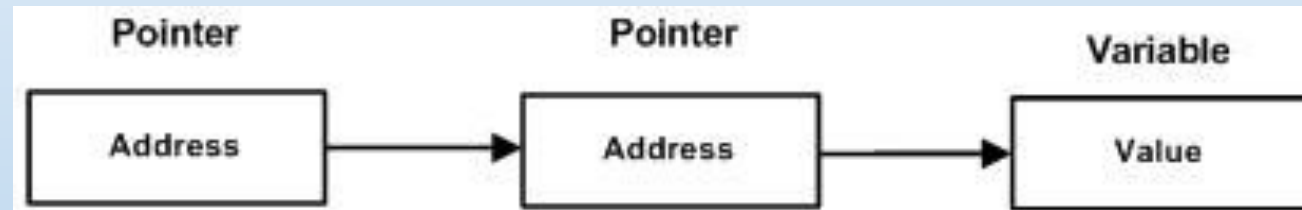
```
}
```

Output:

6 8 20
6 8 20

Pointer to Pointers

- A pointer can also be made to point to a pointer variable (but the pointer must be of a type that allows it to point to a pointer)



Declaration:

- **`int **ptr;`**

Example 1

```
#include <stdio.h>
int main ()
{
    int alpha;
    int *p, **q;
    var = 100;
    p = &var;
    q = &p;
    printf("Alpha = %d\n", alpha );
    printf("Value at pointer *p = %d\n", *p );
    printf("Value at pointer to pointer **q = %d\n", **q);
    return 0;
}
```

Output:

Alpha=100

Value at pointer *p = 100

Value at pointer to pointer **q = 100

Example 2

```
#include<stdio.h>
int main()
{
    int var;
    int *ptr, **ptr2;
    var = 100;
    ptr = &var;
    ptr2 = &ptr;
    printf("Var = %d and Address of Var = %p\n", var, &var);
    printf("Address stored by the ptr = %p and Address of ptr itself = %p\n", ptr, &ptr);
    printf("Address of the pointer stored by the ptr2 = %p\n", ptr2);
    printf("Value at pointer i.e. value at address stored inside the pointer, *ptr = %d\n", *ptr);
    printf("Value at pointer to pointer i.e. value at a pointer address which itself is storing another address, **ptr2 = %d\n", **ptr2);
    return 0;
}
```

```
C:\Users\Amitesh\Desktop\JA x + v
Var = 100 and Address of Var = 0061FF18
Address stored by the ptr = 0061FF18 and Address of ptr itself = 0061FF14
Address of the pointer stored by the ptr2 = 0061FF14
Value at pointer i.e. value at address stored inside the pointer, *ptr = 100
Value at pointer to pointer i.e. value at a pointer address which itself is storing another address, **ptr2 = 100

Process returned 0 (0x0)   execution time : 0.304 s
Press any key to continue.
```

POINTER IN 1D ARRAY

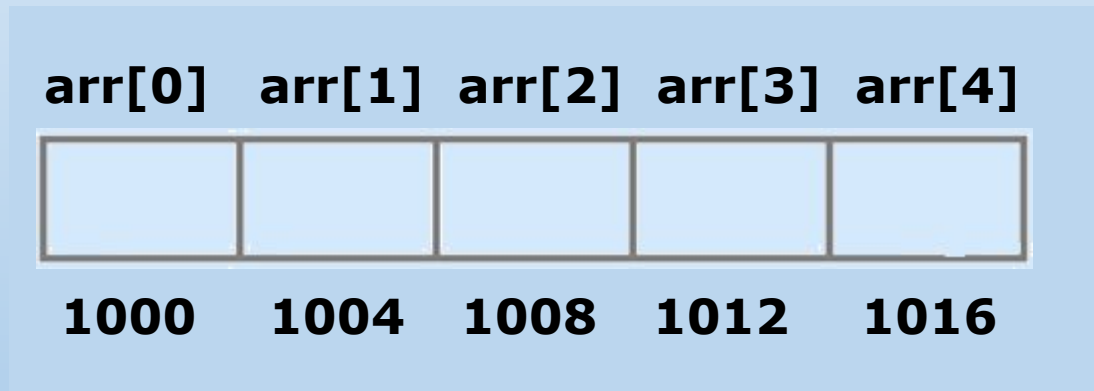
Arrays

When an array is declared, the compiler allocates a sufficient amount of memory to contain all the elements of the array.

The base address which gives the location of the first element is also allocated by the compiler. Suppose, we declare an array **arr**.

```
int arr[5] = {1, 2, 3, 4, 5};
```

Assuming that the **base address** of **arr** is **1000** and each integer requires 4 bytes. Then 5 elements are stored as follows.



Here, **arr** will give a base address, which is a constant pointer pointing to the element **arr[0]**. Therefore, arr contains the address of arr[0] i.e. 1000

arr is equal to &arr[0] //by default

Pointer to array

We can declare a pointer of type int to point to the array **arr**.

```
int *p;
```

```
p = arr;
```

```
or p = &arr[0]; //both statements are equivalent
```

Now, we can access every element of array **arr** using **p++** to move from one element to another.

Note: We cannot decrement a pointer once incremented **p--** - won't work.

Pointer to Array

We can use a pointer to point to an Array. Then use that pointer to access that array.

```
int i;  
int arr[5] = {1, 2, 3, 4, 5};  
int *p = arr;  
for (i=0; i<5; i++)  
{  
    printf("%d", *p);  
    p++;  
}
```

Note: pointer ***p** prints all the values that are stored in the array **arr** one by one.

Replacing **printf("%d", *p);** statement in the previous program, with below statements. Let's observe the result.

`printf("%d", arr[i]);` prints array by incrementing index

`printf("%d", i[arr]);` prints array by incrementing index

`printf("%d", arr + i);` prints address of all the array elements

`printf("%d", *(arr + i));` prints array by incrementing index

`printf("%d", *arr);` prints value of arr[0] only

`arr++;` compile time error, can't change base address of array

Pointer to Array example

```
1  #include<stdio.h>
2  int main()
3  {
4      int i;
5      int arr[] = {1, 2, 3, 4, 5};
6      int *p1 = arr;
7      printf("Displaying pointer p1 to array arr \n\n");
8      for(i=0;i<5;i++)
9      {
10         printf("ITERATION %d : \n\n", i);
11         printf("Value at address containing in p1 i.e *p1= *(&p) = %d\n", p1, *p1 );
12         printf("Alternatively we can write, ");
13         printf("*p1 = *(arr+%d) = *(&p + %d) = *(&p) = %d\n", i, arr, i, (arr+i), *p1 );
14         printf("*p1 = %d = arr[%d]=%d = %d\n\n", *p1, i, arr[i], *(arr + i));
15         p1++;
16     }
17
18     return 0;
19 }
20
```

Logs & others

```
Code::Blocks x Search results x Cccc x Build log x Build messages x CppCheck/Vera++ x CppCheck/Vera++ messages x Cscope x Debugger x D
File Line Message
=== Build file: "no target" in "no project" (compiler: unknown) ===
=== Build finished: 0 error(s), 0 warning(s) (0 minute(s), 0 second(s)) ===
```

C:\Users\Amitesh\Desktop\JA x + v

ITERATION 0 :

Value at address containing in p1 i.e *p1= *(0061FF04) = 1
Alternatively we can write, *p1 = *(arr+0) = *(0061FF04 + 0) = *(0061FF04) = 1
*p1 = 1 = arr[0]=1 = 1

ITERATION 1 :

Value at address containing in p1 i.e *p1= *(0061FF08) = 2
Alternatively we can write, *p1 = *(arr+1) = *(0061FF04 + 1) = *(0061FF08) = 2
*p1 = 2 = arr[1]=2 = 2

ITERATION 2 :

Value at address containing in p1 i.e *p1= *(0061FF0C) = 3
Alternatively we can write, *p1 = *(arr+2) = *(0061FF04 + 2) = *(0061FF0C) = 3
*p1 = 3 = arr[2]=3 = 3

ITERATION 3 :

Value at address containing in p1 i.e *p1= *(0061FF10) = 4
Alternatively we can write, *p1 = *(arr+3) = *(0061FF04 + 3) = *(0061FF10) = 4
*p1 = 4 = arr[3]=4 = 4

ITERATION 4 :

Value at address containing in p1 i.e *p1= *(0061FF14) = 5
Alternatively we can write, *p1 = *(arr+4) = *(0061FF04 + 4) = *(0061FF14) = 5
*p1 = 5 = arr[4]=5 = 5

Process returned 0 (0x0) execution time : 0.193 s
Press any key to continue.

Program

```
#include <stdio.h>
```

```
int main ()
```

```
{
```

```
    int i;
```

```
    int arr[5] = {1, 2, 3, 4, 5};
```

```
    int *p = arr;
```

```
    int i, s = 0;
```

```
    for (i=0; i<5; i++)
```

```
    {    s = s + (*p);
```

```
        p++;
```

```
    }
```

```
    printf("%d", s);
```

```
    return 0;
```

```
}
```

OUTPUT:

15

Note : We can replace statement:

$s = s + (*p);$

by

$s = s + (*(p+i));$

or

$s = s + p[i];$

References:

- <https://www.pdfdrive.com/c-in-depth-books.html>
- <https://www.programiz.com/c-programming/c-pointers>
- <https://www.w3schools.in/c-tutorial/pointers/>

End of Lecture